

Assignment 05

Class Diagram

University Timetable Optimization and Management System

Project: UTOS

Course	Software Engineering
Instructor	Sajid Anwar
Submitted by	Muhammad Hammad Shakeel
Date	May 2026
Assignment	Develop a detailed class diagram with classes, attributes, methods, and multiplicities

Class Catalogue

The catalogue below lists every class in the diagram, grouped by architectural layer, together with its key attributes and methods. Visibility is denoted by – for private and + for public; types follow the standard UML convention attribute: Type.

4.1 Role and Actor Layer

Class	Layer	Key Attributes	Key Methods
Person	Role	– name: String – email: String	+ login() + logout() + getProfile()
Teacher	Role	– employeeNumber: String – maxDailyLoad: Integer	+ submitAvailability() + requestChange() + viewPersonalTimetable() + getSchedule(): List
Student	Role	– registrationNumber: String	+ viewSectionTimetable() + viewExamSchedule() + receiveNotification()
TimetableAdministrator	Role	– authorityLevel: Integer	+ generateTimetable() + publishTimetable() + approveChangeRequest() + manualAdjust() + exportTimetable() + compareVersions()
DepartmentCoordinator	Role	– responsibilityArea: String	+ reviewTimetable() + approveChangeReq() + validateDeptData() + confirmPreferences()
FacilityManager	Role	– responsibilityArea: String	+ manageRoom() + viewUtilization() + updateRoomStatus()
SystemAdministrator	Role	(inherits Person)	+ manageUsers() + assignRole(user, role) + viewAuditLogs() + configureSystem()

4.2 Academic Structure Layer

Class	Layer	Key Attributes	Key Methods
University	Academic	– name: String – campusName: String	+ getFaculties(): List + getDepartments(): List
Faculty	Academic	– name: String	+ getDepartments(): List

Class	Layer	Key Attributes	Key Methods
			+ getCourses(): List
Department	Academic	– name: String – code: String	+ getTeachers(): List + getSections(): List + getCourses(): List + getTimetables(): List
AcademicTerm	Academic	– name: String – startDate: Date – endDate: Date	+ getTimeslots(): List + getHolidays(): List + isActive(): Boolean
Course	Academic	– code: String – title: String – creditHours: Integer	+ getOfferings(): List + getDepartment(): Department
StudentSection	Academic	– name: String – program: String – semester: Integer – batch: String – size: Integer	+ getStudents(): List + getOfferings(): List

4.3 Scheduling Core Layer

Class	Layer	Key Attributes	Key Methods
CourseOffering	Scheduling	– weeklySessions: Integer – requiredRoomType: String – sessionDuration: Integer	+ getSessions(): List + getTeacher(): Teacher + getSection(): StudentSection + getTerm(): AcademicTerm
Timetable	Scheduling	– name: String – status: TimetableStatus – qualityScore: Float – hardConflictCount: Integer – softPenalty: Float – unplacedSessionCount: Integer – createdAt: DateTime	+ generate(): void + publish(): void + getConflicts(): List + getQualityReport(): Report + compareTo(other): Diff + export(format): File
ClassSession	Scheduling	– sessionNumber: Integer – status: SessionStatus – locked: Boolean	+ assignRoom(room): void + assignTimeslot(ts): void + lock(): void + unlock(): void + getConflicts(): List
Timeslot	Scheduling	– day: DayOfWeek – startTime: Time – endTime: Time – isMorning: Boolean – isLastSlot: Boolean	+ getDuration(): Integer + clashesWith(other): Boolean

Class	Layer	Key Attributes	Key Methods
TeacherAvailability	Scheduling	– isAvailable: Boolean – note: String	+ getTimeslot(): Timeslot + getTeacher(): Teacher
SchedulingPolicy	Scheduling	– name: String – policyType: PolicyType – priority: Integer – weight: Float – isHardConstraint: Boolean – isEnabled: Boolean	+ apply(session): Boolean + getPenalty(session): Float + validate(): Boolean
ChangeRequest	Scheduling	– requestType: String – reason: String – urgency: UrgencyLevel – preferredAlternative: String – coordinatorNote: String – status: RequestStatus – createdAt: DateTime	+ approve(note): void + reject(note): void + getSubmitter(): Person
Holiday	Scheduling	– name: String – date: Date – reason: String	+ isOnDate(d: Date): Boolean

4.4 Facility Layer

Class	Layer	Key Attributes	Key Methods
Building	Facility	– name: String – location: String	+ getRooms(): List + getRoomCount(): Integer
Room	Facility	– code: String – floor: Integer – capacity: Integer – roomType: RoomType – features: String – maintenanceStatus: Boolean	+ isAvailable(timeslot): Boolean + getUtilization(): Float

4.5 System-Wide Support Layer

Class	Layer	Key Attributes	Key Methods
TimetableVersion	System	– versionNumber: Integer – createdAt: DateTime – changeSummary: String	+ compareTo(other): VersionDiff + getChangedEntries(): List
Notification	System	– message: String – notificationType: NotifType – createdAt: DateTime – read: Boolean	+ markRead(): void + getRecipient(): Person

Class	Layer	Key Attributes	Key Methods
AuditLog	System	– entityType: String – action: String – oldValue: String – newValue: String – timestamp: DateTime – performedBy: String	+ getActor(): Person

5. Relationships and Multiplicity

The table below catalogues every relationship in the class diagram with its type and multiplicity. UML conventions are applied throughout: 1 (exactly one), 0..1 (zero or one), 0..* (zero or more), 1..* (one or more).

Relationship	Type	Multiplicity	Notes
Person → Teacher, Student, TimetableAdministrator, DeptCoordinator, FacilityManager, SystemAdministrator	Inheritance	1 : 1 (IS-A)	Person is abstract; login/logout shared
University → Faculty	Association	1 to 1..*	University has at least one Faculty
Faculty → Department	Association	1 to 1..*	Faculty has at least one Department
Department → Teacher	Association	1 to 0..*	Department employs zero or more Teachers
Department → StudentSection	Association	1 to 0..*	Sections belong to one Department
Department → Course	Association	1 to 0..*	Department owns courses it offers
Department → Timetable	Association	1 to 0..*	Department has its own timetables per term
AcademicTerm → CourseOffering	Association	1 to 0..*	Offerings are scoped to a term
AcademicTerm → Timeslot	Association	1 to 1..*	Term defines its available timeslots
AcademicTerm → Holiday	Association	1 to 0..*	Term may have zero or more holidays
AcademicTerm → Timetable	Association	1 to 0..*	Multiple timetable drafts per term allowed
Course → CourseOffering	Association	1 to 0..*	Course description reused across offerings
Teacher → CourseOffering	Association	1 to 0..*	Teacher may be assigned to multiple offerings

Relationship	Type	Multiplicity	Notes
StudentSection → CourseOffering	Association	1 to 0..*	Section attends multiple course offerings
StudentSection → Student	Association	1 to 0..*	Section contains enrolled students
Teacher → TeacherAvailability	Association	1 to 0..*	Teacher declares availability per timeslot
TeacherAvailability → Timeslot	Association	1 to 1	Each availability record covers one timeslot
Timetable → ClassSession	Composition	1 to 0..*	Sessions exist only within a timetable
CourseOffering → ClassSession	Association	1 to 1..*	At least one session per offering per week
ClassSession → Timeslot	Association	1 to 0..1	0..1 allows unplaced sessions
ClassSession → Room	Association	1 to 0..1	0..1 allows unplaced sessions
Timetable → SchedulingPolicy	Association	1 to 0..*	Timetable is guided by active policies
Timetable → TimetableVersion	Association	1 to 1..*	Every saved timetable has at least one version
Building → Room	Composition	1 to 1..*	Rooms exist inside a building
FacilityManager → Room	Association	1 to 0..*	Manager is responsible for rooms
Person → ChangeRequest	Association	1 to 0..*	Any person can submit change requests
DeptCoordinator → ChangeRequest	Dependency	1 to 0..*	Coordinator reviews requests
Timetable → Notification	Dependency	1 to 0..*	Publish/change events trigger notifications
TimetableAdministrator → AuditLog	Dependency	1 to 0..*	Admin actions are recorded

5.1 Multiplicity Notation Reference

Notation	Meaning
1	Exactly one — the relationship end is mandatory and unique
0..1	Zero or one — optional reference (e.g., ClassSession may be unplaced without a Room)

Notation	Meaning
0..*	Zero or more — optional collection (e.g., a Department may have zero or more Teachers)
1..*	One or more — at least one element is required (e.g., a Building must contain at least one Room)

6. Enumerations

Enumerations are used as attribute types throughout the design to make invalid states unrepresentable and to align the design with the constraint configuration and timetable status surfaces in the UI.

Enumeration	Values
TimetableStatus	DRAFT, UNDER_REVIEW, PUBLISHED, ARCHIVED, REGENERATING
SessionStatus	PLACED, UNPLACED, LOCKED
RoomType	LECTURE_HALL, SEMINAR_ROOM, LAB, EXAM_ROOM, AMPHITHEATER
RequestStatus	PENDING, UNDER_REVIEW, APPROVED, REJECTED, WITHDRAWN
UrgencyLevel	LOW, MEDIUM, HIGH, CRITICAL
PolicyType	HARD_CONSTRAINT, SOFT_PREFERENCE

7. Key Design Decisions

The following decisions drove the structure of the class diagram. Each entry records the decision and the rationale that motivated it, so that downstream implementation choices can be traced back to a deliberate design intent.

Decision	Rationale
Person as abstract base class	Centralises identity (name, email, login, logout) and avoids duplication across six actor types. All role-specific classes simply extend Person, so a single authentication and profile contract serves every user role.
ClassSession with 0..1 Room and 0..1 Timeslot	Allows unplaced sessions to exist as valid objects inside a draft timetable. This is essential for partial generation, conflict reporting, and the manual-adjustment workflow, where the administrator may temporarily detach a session from its slot.
CourseOffering separate from Course	Course is a reusable description; CourseOffering ties it to a specific term, teacher, and section. The same course can be offered multiple times across terms or sections without duplicating its description.

Decision	Rationale
Timetable composes ClassSession	ClassSession is meaningful only inside a Timetable. The filled-diamond composition signals that sessions are deleted when the parent timetable is removed — they cannot float independently.
TimetableVersion as a separate class	Supports UC15 (Compare Timetable Versions) and provides an audit trail for timetable changes over time. Versioning is orthogonal to the timetable itself, so it lives in its own class rather than as fields on Timetable.
AuditLog as a standalone class	Decouples logging from domain classes. Any entity change can be recorded without modifying the entity itself, satisfying the cross-cutting auditability requirement from the SRS.
SchedulingPolicy carries weight and isHardConstraint	Unifies hard and soft constraint configuration into one class, matching the constraint screen in the UI. The isHardConstraint flag determines whether the policy is strictly enforced or merely penalised in the optimisation objective.
Enumerations over raw Strings	TimetableStatus, SessionStatus, RoomType, and the others prevent invalid attribute values and make the design directly implementation-ready in statically typed languages.

8. Class Diagram

The full class diagram is reproduced below. It is organised into six horizontal bands corresponding to the architectural layers identified in the catalogue:

- Roles and people — the Person hierarchy at the top of the diagram
- Academic structure — University, Faculty, Department, AcademicTerm, Course, StudentSection
- Scheduling core — CourseOffering, Timetable, ClassSession, Timeslot, TeacherAvailability
- Policy and constraints — SchedulingPolicy, Holiday
- Facility — Building, Room
- System-wide — ChangeRequest, Notification, TimetableVersion, AuditLog, and the enumerations

Notation legend: a hollow-triangle arrow denotes inheritance; a filled diamond denotes composition (strong life-cycle ownership); a plain solid line with multiplicity labels denotes association; a dashed arrow denotes a dependency (behavioural coupling without ownership).

9. Traceability to Use Cases

The table below provides a sample of the traceability between the expanded use cases and the methods in the class diagram. Every use case in UC00–UC22 was checked, and the most representative mappings are listed here.

UC ID	Use Case	Primary Class(es)	Primary Method(s)
UC01	Login / Authenticate	Person	login(), logout()
UC03	Submit Teacher Availability	Teacher, TeacherAvailability	submitAvailability()
UC05	Manage Rooms	FacilityManager, Room	manageRoom(), updateRoomStatus()
UC06	Configure Scheduling Policy	SchedulingPolicy	apply(), validate()
UC08	Generate Timetable	Timetable, ClassSession	generate(), assignRoom(), assignTimeslot()
UC09	Review Timetable Conflicts	Timetable, ClassSession	getConflicts()
UC10	Manually Adjust Timetable	TimetableAdministrator, ClassSession	manualAdjust(), lock(), unlock()
UC11	Publish Timetable	Timetable, Notification	publish(), markRead()
UC13	Submit Change Request	Teacher, ChangeRequest	requestChange()
UC14	Review / Approve Change Request	DepartmentCoordinator, TimetableAdministrator, ChangeRequest	approve(), reject()
UC15	Compare Timetable Versions	Timetable, TimetableVersion	compareTo(), getChangedEntries()
UC17	Export Timetable	TimetableAdministrator, Timetable	export(format)
UC19	View Personal Timetable	Teacher, Student	viewPersonalTimetable(), viewSectionTimetable()
UC21	View Audit Logs	SystemAdministrator, AuditLog	viewAuditLogs(), getActor()
UC22	Manage Users / Assign Role	SystemAdministrator	manageUsers(), assignRole(user, role)

10. Conclusion

The class diagram presented in this document captures the complete static structure of the UTOS at the design layer. Twenty-six domain and system classes, organised into six architectural layers and connected by inheritance, composition, association, and dependency relationships, provide a direct blueprint for implementation.

Every actor responsibility identified in the use cases maps to at least one method; every entity in the domain model maps to a class; every multiplicity from the domain model is preserved on its relationship end; and every cross-cutting concern (auditing, notification, versioning) is modelled as a standalone class so that it can be implemented without polluting the core domain entities.

The diagram is therefore ready to be translated into source code: each class becomes a class declaration, each multiplicity becomes a typed collection or reference, each method becomes an operation signature, and each enumeration becomes a language-level enum or sealed type.